ENGINEERING PORTFOLIO

Ariadne Colina-Valeri

Hi! My name is Ariadne, and I'm a Junior studying Electrical Engineering at Tufts University. I have experience in signal processing, embedded systems and machine learning, from restoring RF signals with a Wiener filter to building a real-time audio effects board and an autonomous robot. I'm looking for internships in those fields.

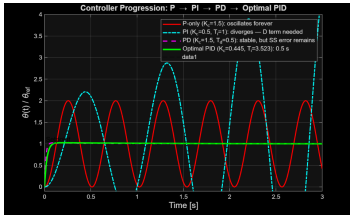


ariadnecolina@gmail.com
(813) 847-5642

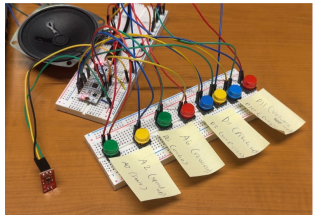
SELECTED PROJECTS



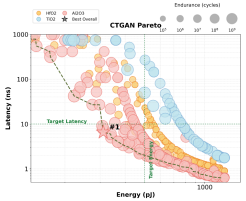
Autonomous Robot
page 2



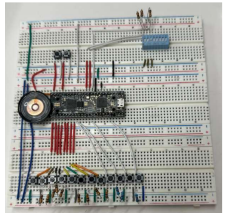
UAV Pitch Control
page 3



DJ Board on an STM32
page 4



CTGAN for RRAM
page 5



FPGA Synthesizer
page 6

PROJECT 1 | IN PROGRESS | JUNIOR DESIGN | SEPT. 2026 TO PRESENT

Autonomous ESP32 Robot

THE CHALLENGE

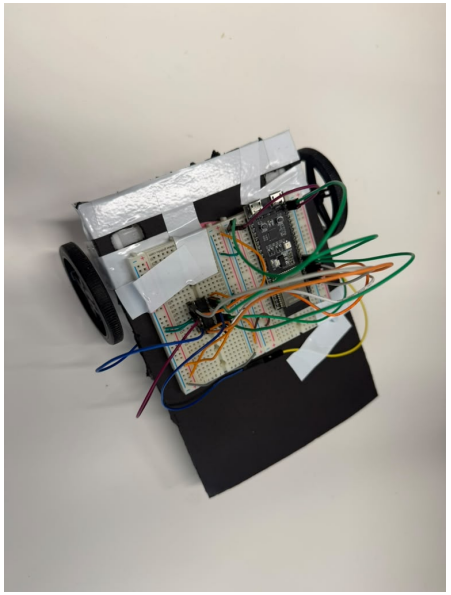
The robot has to follow a colored lane and avoid obstacles with no one controlling it, and accept commands only from our own team.

WHAT I'M DOING

I'm building a 7-state state machine on the ESP32 that authenticates every websocket command with our team key and rejects the rest. For obstacles, an IR LED and photodetector measure reflected light, and I subtract the room's background light from each reading. For the lane, I filter noise from the color sensor before the robot decides which color it is on.

NEXT

A ladybug-shaped body I'm designing in SolidWorks, and a custom ESP32 PCB with parts chosen from datasheets to replace the breadboard wiring.



The robot so far, wired on a breadboard around the ESP32



The ladybug body I'm designing in SolidWorks

UAV Pitch Control and RF Signal Restoration

THE CHALLENGE

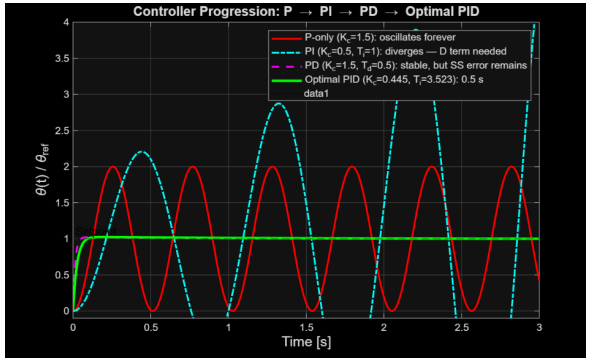
A UAV that shakes while collecting RF signals blurs them, so it has to stay level before the signals can be trusted.

WHAT I DID

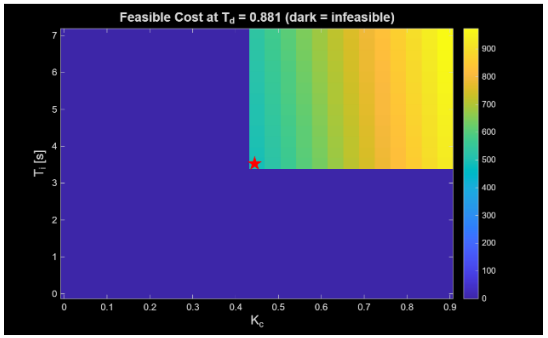
First, I used Routh-Hurwitz analysis to prove that P and PI controllers can never keep the UAV stable, so it needed a full PID controller. I then tested 15,625 combinations of PID settings to find the best one. Next, I restored a blurred RF signal with a Wiener filter in MATLAB. Finally, I showed how the two parts connect: the longer the controller takes to settle, the longer the signal stays blurred.

THE RESULT

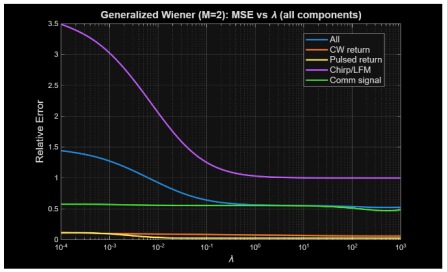
The PID controller rises in 52ms with only 2.6% overshoot and an 88.3° phase margin. The Wiener filter cut the error by 77% compared with a direct inverse, and a controller that settles in 2s instead of 0.5s makes the recovery 1.5× worse.

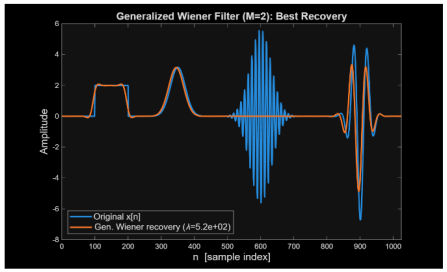


P, PI, PD and PID controllers



Cost of each gain pair; the star is the optimum





Wiener filter error (left) and the restored RF signal over the original (right)

DJ Board on an STM32

(Audio Recorder and Effects Processor)

THE CHALLENGE

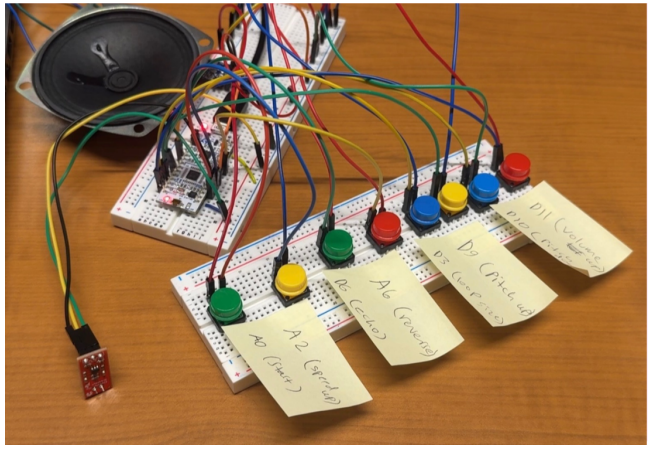
Seven real-time audio effects had to fit in the STM32's 64KB of on-chip memory, and the first playback came out with a high-pitched hiss and clicks.

WHAT I DID

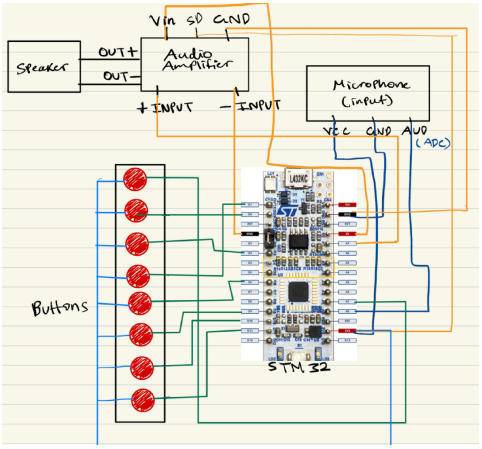
To remove the hiss, I used a 4kHz timer interrupt to record and play every sample at the same rate, and lengthened the ADC sampling time to 12.5 cycles so the microphone signal settles before each reading. The clicks came from sudden jumps in the speaker signal, so I held the output at mid-rail between clips and faded each clip in and out over 256 samples (64ms). Dropping from 8kHz to 4kHz stretched each recording from 2 to 5 seconds.

THE RESULT

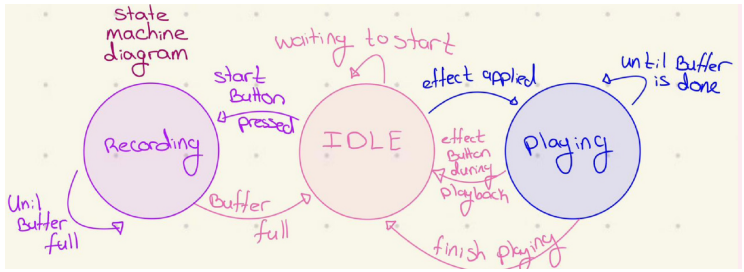
The board records 5 seconds of 12-bit audio in a 40KB buffer, plays all 7 effects (echo, reverse, speed, loop size, pitch up, pitch down, volume), and its playback matched the original recording on the oscilloscope.



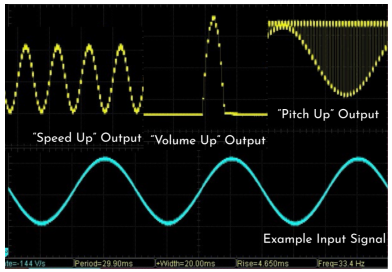
The board



Hardware: wiring diagram



Software: the state machine



Oscilloscope captures of the effects

STM32: every sample moves on a 4kHz timer



CTGAN Design Discovery for RRAM

THE CHALLENGE

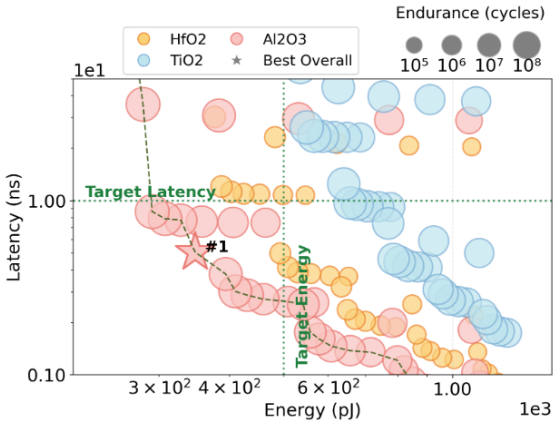
The published framework I built on used a CVAE, an AI model that averages its training data. Averaging hides the unusual designs, and those are where the best performance is.

WHAT I DID

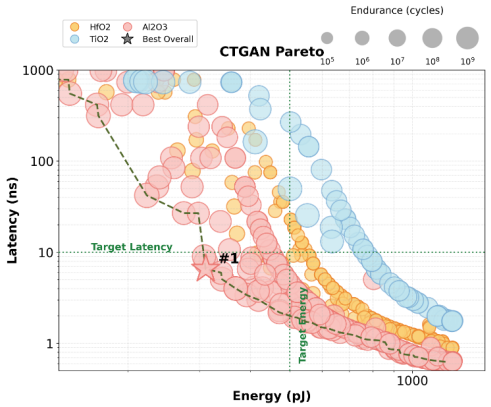
I replaced it with a CTGAN, a model that searches those unusual designs more aggressively. To keep it from inventing designs that cannot physically exist, I used the framework's physics model (PINN) to reject them. The CTGAN failed with 500 training samples and was still noisy with 2,500, so I grew the data to 5,000, where it worked for all 3 materials (Al_2O_3, HfO_2 and TiO_2).

THE RESULT

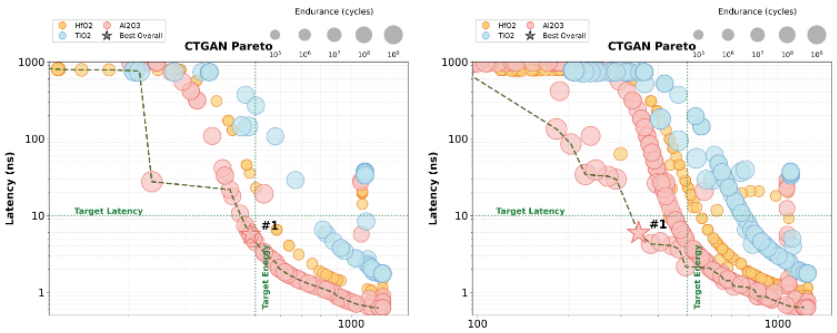
My best Al_2O_3 design lasted 25% longer than the published one (4.84 billion cycles against 3.85 billion) and used 11% less energy, though it switched a little slower (6.57ns against 4.07ns).



Published CVAE results



My CTGAN with 5,000 samples



The CTGAN with 500 samples (left) and 2,500 (right)

FPGA Audio Synthesizer

THE CHALLENGE

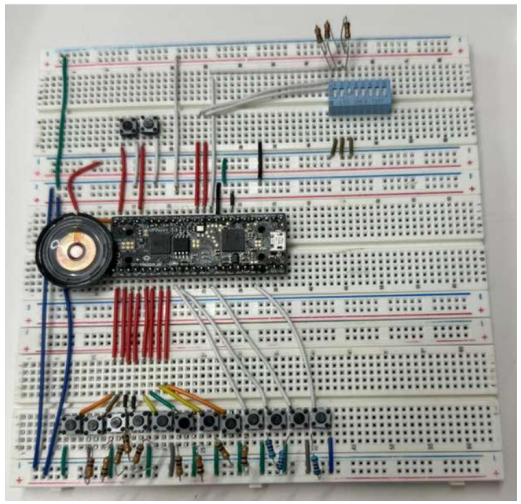
Each octave doubles a note's frequency, and my first design calculated every note by dividing on the chip, which needed over 250% of the FPGA's space. Chords of more than 2 notes, or in low octaves, turned into static.

WHAT I DID

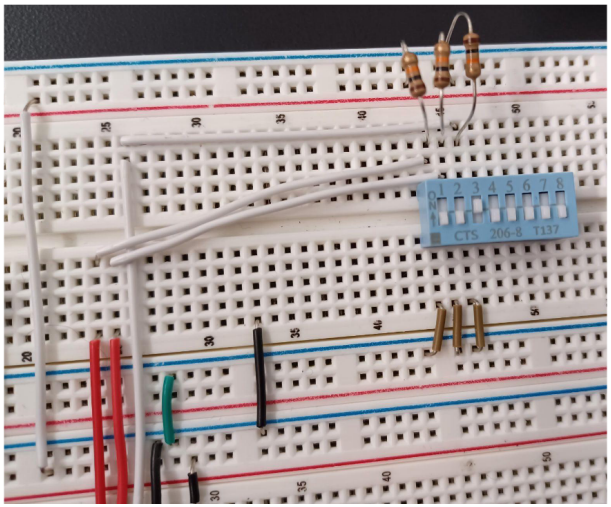
First, a tuner showed every note playing an octave too low, which meant the clock was running at twice the rate the code assumed, so I rebuilt the scale for the real clock. To fit the design on the chip, I calculated one octave by hand and got the others with bit shifts, since each octave doubles the frequency. I also widened the period counter so notes below 92Hz could play. To fix the static, I plotted chords in Desmos, then mixed the notes with PWM on a 3-bit bus, the best balance of volume and clarity.

THE RESULT

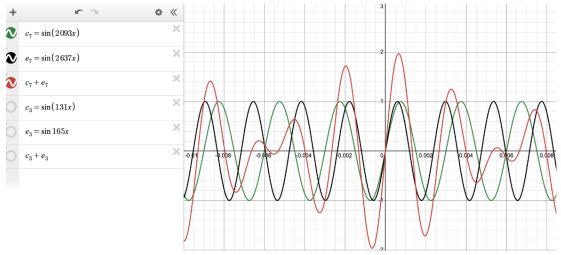
The keyboard plays all 12 notes across 8 octaves (16Hz to 8kHz), chords of 5 to 6 notes in the lowest octave, and every note at once in the highest.



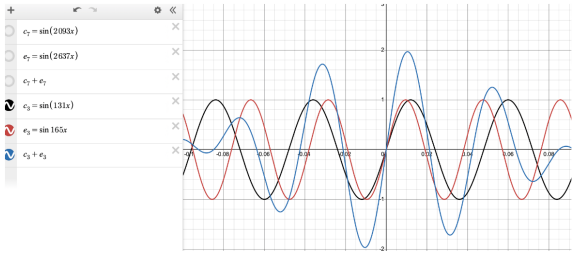
The complete board



The switch that selects the octave



C + E summed in octave 7 (red)



C + E summed in octave 3 (blue)